Drink Recommender

Architecture and design decisions - SplitEasy

September 2026

What it does

You give it four things - which state you are in, who you are drinking with, how many people and what budget - and it answers the question you actually have: which bottles do I walk out of the shop with, and what will that cost each of us.

The one design decision that matters

There is no language model anywhere in this feature.

That was a deliberate choice, not a limitation. A model asked to recommend drinks will happily invent a price for a brand in a state it has never seen, and the answer will read exactly like a real one. For a feature whose entire job is money, that failure is worse than useless - it is confidently wrong.

So every number here is either looked up in a table with a citation, or computed in code from your own recorded spending. Prices are read, arithmetic is done in Python, and the ranking is a sort you can read and argue with. Every figure on screen can be traced to a source URL or one of your own expenses.

The two data sources

The recommender stands on two legs, and they do different jobs.

1. Price table	backend/liquor_prices.py - a hand-curated table scraped from public state price listings. Answers 'what does this bottle cost here'.
2. Your history	The expenses already in the app. Answers 'what do these particular people actually drink, and what do they normally spend'.

The second is the one a generic app cannot copy. Knowing that you and Anubhav have had 18 sessions averaging Rs 1,458 and keep buying Old Monk is worth more than any amount of general knowledge about whisky.

Request flow

```
BROWSER                     FastAPI                     DATA
---------------             -------------------         -----------------

Recommend.jsx
  state, people,
  budget, strength,   --->  GET /recommend/
  names[]                     |
                              | 1. auth: who is calling?
                              |    (token -> User)
                              |
                              | 2. _history(caller, names)  --->  Postgres
                              |    scoped to groups you               (expenses)
                              |    are a member of
                              |
                              | 3. for_state(state)         --->  liquor_prices
                              |    rows for that state                (static table)
                              |
                              | 4. _pick(): bottles + price
                              |    _best_combo() per brand
                              |
                              | 5. across_sizes(): the same
                              |    basket in UP / DL / GGN
                              v
render cards        <---  { history, picks[], compare[] }
```

Step 4 - choosing the bottles

Spirits in India are sold in three sizes and everybody names them the same way. The recommender only ever suggests these:

180 ml	quarter
375 ml	half
750 ml	full

An evening's demand is estimated in millilitres - roughly one quarter a head for a normal night - but that number is never shown, because nobody buys 540 ml. It is a target that gets converted into whole bottles.

_best_combo() then solves a tiny three-coin problem: which combination of quarters, halves and fulls covers the target most cheaply. With three denominations and small counts, brute force over the grid is exact and instant - no need for anything cleverer.

```
3 people, normal  ->  target 540 ml

   1 full   750 ml   Rs 520     <- chosen: cheapest cover
   1 half + 1 qtr    555 ml     Rs 555
   3 qtr    540 ml   Rs 570
```

Overshoot is allowed up to a half-bottle. A tighter cap looked more principled and was wrong in practice: it rejected a single full for three people, which overshoots by 210 ml and is obviously what anyone would buy.

Step 5 - the NCR comparison

Alcohol is a State subject in India. Every state sets its own excise duty and its own MRP, so the same bottle genuinely costs different amounts a short drive apart. That is not a rounding difference - it is often hundreds of rupees.

```
Royal Stag 750 ml        UP  650-720    Delhi 550-800    Gurugram 480-550
Blenders Pride 750 ml    UP  680-780    Delhi 550-800    Gurugram 730
Old Monk 750 ml          UP  520        Delhi 350-420    Gurugram  -
```

So every suggestion is priced in all three NCR regions and the cheapest is highlighted. A dash means that region publishes no price for that exact bottle and size. It is never a number carried across from a neighbouring state, even though that would fill the grid more neatly.

A region is only priced when it has every size in the basket. Pricing half a basket and estimating the rest would produce a total that reads as real and is not.

Honesty rules in the price table

- Every row cites a source URL and the month it was read.
- Nothing is interpolated between states. A missing state is a dash, not a guess.
- Where a source published a range, the range is kept rather than averaged away.
- Haryana sets a minimum selling price, not a fixed MRP, so shops legally differ and two honest sources disagreed. Those rows span both figures instead of one being picked and presented as the price.

Current coverage: Delhi, Gurugram (Haryana), Uttar Pradesh and Maharashtra. Running 'python liquor_prices.py' prints coverage and every source, so the gaps are visible rather than implied.

Privacy

History is computed only from groups the caller is a member of. Naming somebody you do not share a group with returns nothing about them - the endpoint cannot be used to read another person's drinking habits, in the same way the rest of the API cannot be used to read their balances.

Ranking

The sort is three keys, in this order:

1. Familiar first	Brands this set of people has actually bought before.
2. Right amount	Closest total volume to what this many people would get through.
3. Cheapest	Then price, ascending.

History only appears on screen once you have named somebody. With nobody named it was the caller's own drinking across every group, shown as 'sessions together' with no one to have had them with. The numbers still rank the suggestions in that case - they just are not presented as shared history.

Files

backend/liquor_prices.py	The price table, its sources, and the cross-region lookups. Pure data plus small helpers - no database, no framework.
backend/routers/recommend.py	Two endpoints. GET /recommend/meta lists the states we have prices for; GET /recommend/ does the work. Holds the history scan, the bottle solver and the ranking.
frontend/src/pages/Recommend.jsx	The form and the result cards, including the three-region price strip.
frontend/src/components/BottomNav.jsx	The raised centre key routes here.

Known limits

- Uttar Pradesh coverage is thin - 8 brands - because UP publishes PDFs rather than anything machine-readable. Real shop prices would fix this fastest.
- Prices drift with every excise year, and shops vary within a state. These are indicative, which is what the source dates are for.
- Brand matching across regions is on name and size. A brand written differently by two sources will not match, and shows as a dash rather than a wrong price.
- The demand model - about a quarter-bottle a head - is a starting point, not science. The budget is the real control.

What it would take to add a state

Add rows to liquor_prices.py with a real source, and it appears in the dropdown automatically. Nothing else changes: the solver, the comparison and the ranking are all driven off the table. If a state should join the NCR comparison, add it to the NCR tuple.